Enabling MXFP8 Training on NVIDIA RTX 5090 (SM 12.0)

Building from Source, Fixing Correctness, and Performance Analysis

---

1. Introduction

This report documents the end-to-end process of enabling MXFP8 (Microscaling FP8) training on the NVIDIA RTX 5090 consumer GPU (Blackwell, SM 12.0). The work spans building the entire software stack from source, fixing correctness bugs in Transformer Engine, analyzing performance characteristics, and benchmarking four FP8 recipes on a 561M-parameter language model.

Project Goals

1. Build PyTorch and Transformer Engine from source with CUDA 13.0 / SM 12.0 support
2. Enable MXFP8BlockScaling on RTX 5090, which was software-blocked in TE
3. Fix backward pass correctness (weight gradients were completely wrong)
4. Analyze performance across BF16, DelayedScaling, Float8BlockScaling, and MXFP8BlockScaling
5. Identify and close the MXFP8 vs Float8BlockScaling performance gap

Environment

Component	Version
GPU	NVIDIA RTX 5090 (SM 12.0, 32 GB VRAM)
CUDA Toolkit	13.0.1
PyTorch	2.11.0a0+git5e4d6fb (compiled from source)
Transformer Engine	2.13.0.dev+c93ca27a (compiled from source)
cuBLASLt	13.0.2
Python	3.10.12
GCC	12
Dion (Muon optimizer)	0.1.0 (NorMuon)

Model Configuration

Parameter	Value
Architecture	Decoder-only transformer (nanochat)
Parameters	561M
Layers	20
Hidden size	1280
Attention heads	10
FFN hidden	5120 (4x)
Activation	Squared ReLU
Normalization	RMSNorm, QK-norm
Positional encoding	RoPE
Vocab size	65536
Sequence length	2048
Logit softcapping	30.0 * tanh(logits / 30.0)
Optimizer	NorMuon (hidden weights) + AdamW (embeddings, gains)
Dataset	FineWeb10B (900M training tokens)

---

2. Building the Software Stack from Source

The RTX 5090 (SM 12.0) requires CUDA 13.0 and native SM 12.0 kernel compilation. At the time of this work, no pre-built pip wheels existed for this combination, so both PyTorch and Transformer Engine had to be compiled from source.

2.1 PyTorch Source Build

Source: `/home/nanogpt/pytorch` (nightly, commit `5e4d6fb`)

```
CUDA_HOME=/usr/local/cuda \
TORCH_CUDA_ARCH_LIST="9.0;12.0+PTX" \
MAX_JOBS=32 \
BUILD_TEST=0 \
python setup.py develop
```

Issues encountered and resolved:

1. **CMake version**: System cmake 3.22 was below the required 3.27. Solved by symlinking the pip-installed cmake:

   ```
   ln -sf

   ~/.venv/lib/python3.10/site-packages/cmake/data/bin/cmake ~/.local/bin/cmake
   ```

2. **GCC 12 Internal Compiler Error (ICE)**: GCC crashed compiling `torch/headeronly/macros/Macros.h` lines 201-202. Fixed by patching the `C10_LIKELY` /`C10_UNLIKELY` macros:

   ```
   // Before (causes GCC 12 ICE):
   static_cast<bool>(expr)
   // After:
   !!(expr)
   ```

3. **Architecture list**: `TORCH_CUDA_ARCH_LIST` must include SM 9.0 alongside 12.0 · some internal kernels require a non-PTX architecture target.

2.2 Transformer Engine Source Build

Source: `/home/nanogpt/te/TransformerEngine` (dev branch, 2.13.0.dev)

```
CUDA_HOME=/usr/local/cuda \
NVTE_FRAMEWORK=pytorch \
NVTE_CUDA_ARCHS="90;120" \
CUDNN_PATH=~/.venv/lib/python3.10/site-packages/nvidia/cudnn \
MAX_JOBS=32 \
pip install --no-build-isolation -e
/home/nanogpt/te/TransformerEngine
```

Issues encountered and resolved:

1. **Empty architecture list**: Setting `NVTE_CUDA_ARCHS=120` alone causes cmake to filter out 120 as a "special" architecture, leaving an empty list. Must include at least one non-special arch: `"90;120"`.
2. **CUDA 12 library conflicts**: `nvidia-*-cu12` pip packages provided `libcudart.so.12` alongside the system `libcudart.so.13`, causing:

   ```
   RuntimeError: Multiple libcudart libraries found:
   libcudart.so.12 and libcudart.so.13
   ```

   Resolved by uninstalling all `nvidia-*-cu12` packages and force-installing cu13 equivalents.
3. **ABI coupling**: TE must be rebuilt whenever PyTorch is rebuilt due to ABI compatibility requirements.

2.3 Rebuilding TE After C++ Changes

Two rebuild paths depending on which files were modified:

```
# For PyTorch extension files (gemm.cpp, quantizer.cpp):
cd /home/nanogpt/te/TransformerEngine && \
  PATH=/usr/bin:/usr/local/bin:/usr/local/cuda/bin:$PATH \
  NVTE_FRAMEWORK=pytorch MAX_JOBS=16 \
  python setup.py build_ext --inplace
# For common C++ library (cublaslt_gemm.cu, swizzle.cu):
PATH=/usr/bin:/usr/local/bin:/usr/local/cuda/bin:$PATH \
```

```
  cmake --build /home/nanogpt/te/TransformerEngine/build/cmake
--parallel 16
```

---

3. Background: FP8 Recipes in Transformer Engine

Transformer Engine supports multiple FP8 quantization recipes. Four are benchmarked in this report:

- **BF16 (no FP8)**: Pure bfloat16 training with AMP autocast. Baseline for comparison.
- **DelayedScaling**: The original FP8 recipe. Uses per-tensor scaling factors computed from the previous iteration's amax history. Supported on Hopper (SM 9.0) and Blackwell.
- **Float8BlockScaling**: Block-wise FP8 scaling. Divides tensors into blocks and computes per-block scaling factors. On SM 12.0, internally converted to MXFP8 format at GEMM time via `convert_block_scaling_to_mxfp8_tensor()`.
- **MXFP8BlockScaling**: Native Microscaling FP8. Quantizes tensors directly into the MXFP8 format with E8M0 scaling factors. The format the hardware natively executes.

Key insight: On SM 12.0, Float8BlockScaling and MXFP8BlockScaling execute the same MXFP8 GEMM kernels. The difference lies entirely in how data is prepared before the GEMM.

---

4. Enabling MXFP8 on SM 12.0

Enabling MXFP8 on the RTX 5090 required fixing three distinct blockers: a Python-level software guard, broken backward-pass GEMM layouts, and incompatible pre-swizzled scale formats. This section covers all three.

4.1 Blocker 1: Python Guard in `quantization.py`

TE 2.13.0.dev explicitly blocked MXFP8 on SM 12.0:

```
# Original code:
if get_device_compute_capability() >= (12, 0):
    return False, "not supported on 12.0+ architectures yet"
```

Fix: Removed the SM 12.0 guard from `check_mxfp8_support()` and `get_default_fp8_recipe()`:

```
@functools.lru_cache(maxsize=None)
def check_mxfp8_support() -> Tuple[bool, str]:
    if get_device_compute_capability() >= (10, 0):
        return True, ""
    return False, "Device compute capability 10.0 or higher
required."
```

After this change, `te.is_mxfp8_available()` returned `True` and forward passes worked. However, the backward pass produced completely wrong gradients.

4.2 Symptom: Wrong Gradients

The diagnostic script (`diag_fwd_bwd.py`) showed:

- Forward output cosine similarity vs BF16: ~0.999 (correct)
- **Weight gradient cosine similarity vs BF16: ~0.0** (catastrophically wrong)

4.3 Blocker 2: cuBLASLt TN-Only Constraint

cuBLASLt on SM 12.0 only supports **TN (transposed-A, normal-B) layout** for MXFP8 GEMM. The function `nvte_is_non_tn_fp8_gemm_supported()` returns `false` for SM 12.0.

The three GEMM operations in a transformer layer use different layouts:

Operation	Layout	SM 12.0 Status
Forward (fprop)	TN	Works natively
Backward dgrad	NN	Not supported, needs fallback
Backward wgrad	NT	Not supported, needs fallback

TE had an existing fallback in `CanonicalizeGemmInput()` designed for Float8BlockScaling, but MXFP8 stores data differently:

Property	Float8BlockScaling	MXFP8BlockScaling
`columnwise_data` shape	**[K, M]** (pre-transposed)	**[M, K]** (same as original)
`columnwise_scale_inv`	Pre-swizzled for GEMM	Natural (unswizzled) format

The existing fallback assumed columnwise data was already transposed. When applied to MXFP8 data in [M, K] shape, it produced garbage results.

4.4 Fix: MXFP8 TN Transpose in `gemm.cpp`

File: `transformer_engine/pytorch/csrc/extensions/gemm.cpp` (line ~242)

Added a `transpose_mxfp8_operand` lambda that executes before the existing `swizzle_scales_for_gemm` calls. The logic:

```
Guard: is_mxfp8 && !nvte_is_non_tn_fp8_gemm_supported() && (!transa
|| transb)
```

For each MXFP8 operand that needs conversion (A if not transposed, B if transposed):

1. **Extract columnwise data and scale** from the TensorWrapper
2. **Flatten to 2D**: Collapse all-but-last dims to get `[first_dim, last_dim]`
3. **Transpose data**: `at::from_blob(..., {first_dim, last_dim}).t().contiguous()` produces `[last_dim, first_dim]`
4. **Transpose scale_inv**: columnwise scale shape `[ceil(first_dim/32), last_dim_padded]` becomes `[last_dim_padded, ceil(first_dim/32)]` · matching rowwise scale shape for the transposed data
5. **Create new TensorWrapper**: Set transposed data as rowwise, transposed scales as rowwise scale_inv, with MXFP8 scaling mode
6. **Keep tensors alive**: Store transposed `at::Tensor` objects in a vector to prevent deallocation during the GEMM

After processing both operands, set `transa=true, transb=false` so the downstream code (swizzle + `CanonicalizeGemmInput`) handles it via the standard TN path.

The initial implementation used `.t().contiguous()` for the data transpose. This was later optimized to use `nvte_transpose` (see Section 7).

4.5 Blocker 3: Pre-Swizzled Scales in `quantizer.cpp`

File: `transformer_engine/pytorch/csrc/quantizer.cpp` (MXFP8Quantizer class)

The `gemm.cpp` fix transposes columnwise scales at GEMM time, which requires them to be in natural (unswizzled) format. If `optimize_for_gemm` is enabled, scales are pre-swizzled at quantization time · the wrong format after transposition.

Changed in both `create_tensor()` and `convert_and_update_tensor()`:

```
// Before (broken on SM 12.0):
```

```
const bool with_gemm_swizzled_scales = this->optimize_for_gemm;
// After (fixed):
const bool with_gemm_swizzled_scales =
    this->optimize_for_gemm && nvte_is_non_tn_fp8_gemm_supported();
```

On SM 12.0, `nvte_is_non_tn_fp8_gemm_supported()` returns `false`, so scales remain in natural format. On Hopper (SM 9.0+), the original behavior is preserved.

4.6 Verification

After the fix, `diag_fwd_bwd.py` confirmed correctness:

```
COSINE SIMILARITY of weight gradients vs BF16
  Float8BlockScaling: 0.999611
  MXFP8BlockScaling : 0.999611
```

Both recipes produce **numerically identical** results on SM 12.0.

---

5. Numerical Accuracy

Single `te.Linear(1280, 1280)` layer, batch=64. All metrics vs BF16 reference.

| Metric | Float8BlockScaling | MXFP8BlockScaling | |---|---|---| | Forward cosine similarity | 0.999292 | 0.999292 | | Forward max absolute error | 6.25000 | 6.25000 | | Forward mean relative error | 35.22% | 35.22% | | Backward (wgrad) cosine similarity | 0.999611 | 0.999611 | | Backward max absolute error | 0.00001 | 0.00001 | | Backward mean relative error | 8.73% | 8.73% |

Identical results confirm both recipes execute the same MXFP8 GEMM kernels on SM 12.0.

---

6. Performance Benchmarks

All benchmarks: 561M model, BS=4, block_size=2048, 50 optimizer steps, single RTX 5090. Timing averaged from steps 2-49 (step 1 excluded as warmup). MFU = `6 * num_params * tokens_per_step / (step_time * GPU_PEAK_TFLOPS)` with GPU_PEAK_TFLOPS = 404.

6.1 Throughput Summary

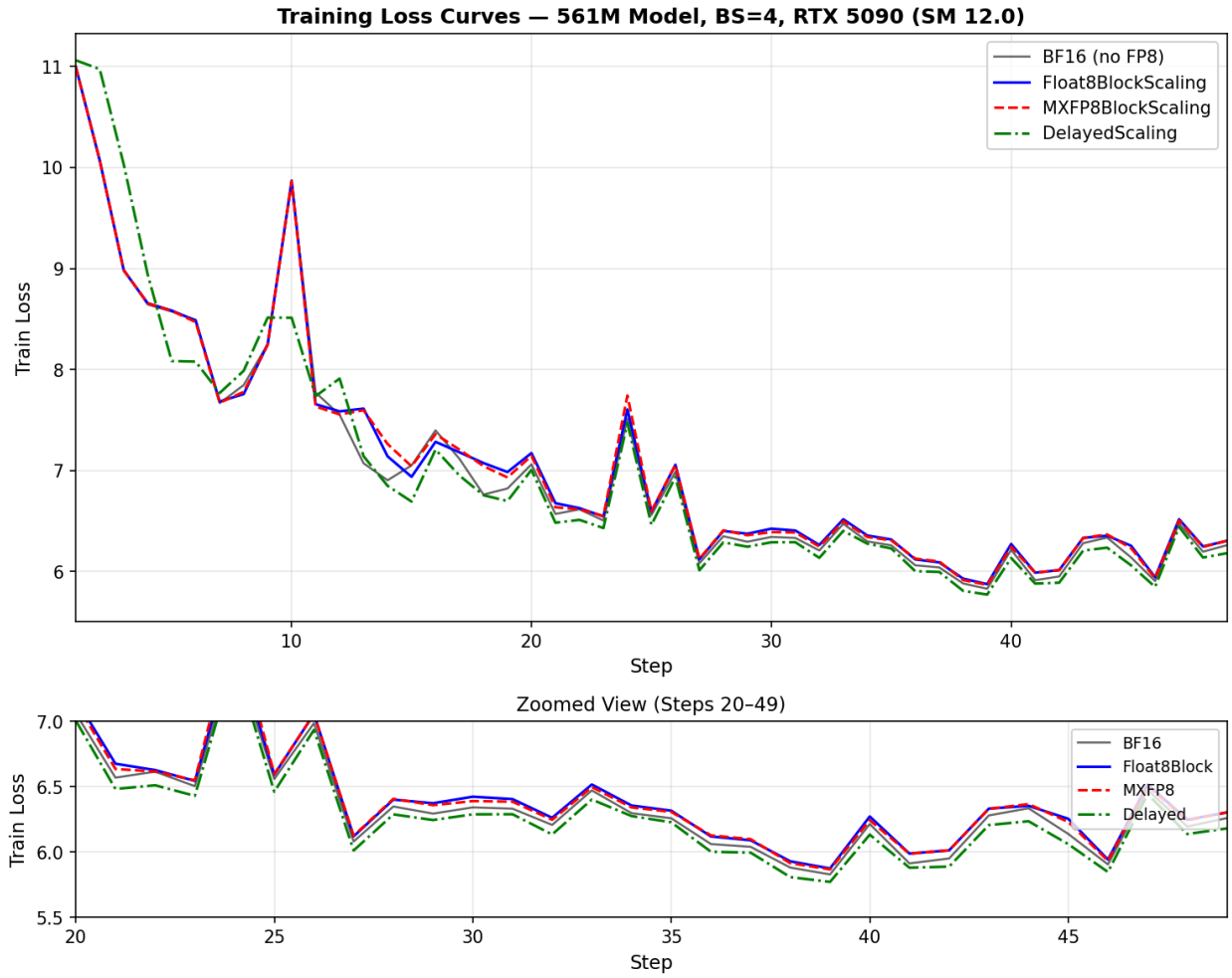
| Recipe | ms/step | Tokens/sec | MFU | Peak Memory | Speedup vs BF16 | |---|---|---|---|---|---|---| | BF16 (no FP8) | ~175 | ~46,900 | ~39.1% | 28.5 GB | 1.00x | | DelayedScaling | ~124 | ~65,700 | ~54.8% | 27.9 GB | 1.41x | | **Float8BlockScaling** | **~113** | **~72,500** | **~60.8%** | **28.9 GB** | **1.55x** | | **MXFP8BlockScaling** | **~116** | **~70,400** | **~58.6%** | **31.2 GB** | **1.51x** |

Note: MXFP8 results reflect the `nvte_transpose` *optimization (Section 7). Before optimization, MXFP8 ran at ~159 ms/step (42.7% MFU).*

6.2 Training Loss Curves

```
Step   BF16     Float8Blk  MXFP8      Delayed
----   ------   --------   ------     -------
   1   10.998   10.999     10.999     11.063
   2   10.060   10.072     10.072     10.975
   3    8.991    8.982      8.982     10.031
   4    8.643    8.656      8.656      8.940
```

5	8.574	8.583	8.581	8.083
10	9.859	9.870	9.866	8.512
15	7.049	6.937	7.040	6.691
20	7.060	7.172	7.145	7.005
25	6.558	6.592	6.597	6.461
30	6.341	6.423	6.389	6.288
35	6.258	6.316	6.306	6.227
40	6.213	6.271	6.243	6.132
45	6.138	6.254	6.229	6.059
49	6.259	6.303	6.303	6.180



Training Loss Curves

All four recipes converge. Float8BlockScaling and MXFP8BlockScaling produce virtually identical loss trajectories (expected: same GEMM kernels on SM 12.0). DelayedScaling converges slightly faster in early steps.

6.3 Validation Loss

At step 40: BF16 = 6.21, Float8Block = 6.35, MXFP8 = 6.24, Delayed = 6.20.

7. Performance Optimization: Closing the MXFP8 Gap

The initial MXFP8 implementation was ~41% slower than Float8BlockScaling (159 ms vs 113 ms). This section traces the root cause and the optimization that closed the gap.

7.1 Initial Profiling: Two Bottlenecks

Nsys profiling of the MXFP8 path revealed two major costs:

Kernel	Calls/step	Avg time	Total/step	% GPU time
`elementwise_kernel` (uint8 copy)	91,854	89 µs	8,171 ms	**29.5%**
`quantize_mxfp8_kernel` (bidimensional)	3,780	153 µs	579 ms	2.1%

The dominant cost was **not** the quantization kernel — it was **280 GB of uint8 tensor copies** that did not exist in Float8BlockScaling at all.

7.2 Root Cause: Inefficient Transpose in Backward GEMMs

The SM 12.0 TN fix in `gemm.cpp` transposes MXFP8 columnwise data `[M,K]` → `[K,M]` for each backward GEMM (dgrad NN and wgrad NT layouts). The original implementation used:

```
auto data_transposed = at::from_blob(..., {M, K},
uint8_opts).t().contiguous();
```

This dispatched PyTorch's **generic** `direct_copy_kernel_cuda`, which performs an element-by-element copy with non-coalesced memory access — achieving only ~2% of peak memory bandwidth.

Per step (63 micro-batches):

- 30,132 uint8 clone operations
- 280 GB of data copied
- ~43 ms wall-clock overhead — nearly the entire MXFP8 vs Float8Block gap

Float8BlockScaling avoids this entirely because its `columnwise_data` is stored pre-transposed as `[K,M]`, so `convert_block_scaling_to_mxfp8_tensor` reuses the data pointer with zero copy.

7.3 Discovery: Tracing the Copies

The copies were invisible to standard Python profiling because they occurred inside the C++ extension (`tex.generic_gemm`). A custom `TorchDispatchMode` tracker on `aten.clone.default` revealed:

```
30,132 uint8 clone calls, all from gemm.py:197:general_gemm
Sources: layernorm_mlp.py backward (fc1/fc2 wgrad+dgrad), linear.py
backward, layernorm_linear.py backward
```

7.4 Fix: TE's Optimized `nvte_transpose`

Transformer Engine already includes an optimized transpose kernel (`nvte_transpose` in `transpose.cu`) that uses vectorized tiled loads/stores for near-peak memory bandwidth. The fix replaces the generic `.t().contiguous()` with this kernel:

```
// Before (slow — generic elementwise kernel, ~2% peak BW):
auto data_transposed = at::from_blob(col_data.data_ptr,
    {first_dim, last_dim}, uint8_opts).t().contiguous();
// After (fast — TE optimized transpose, near-peak BW):
auto data_transposed = at::empty({last_dim, first_dim},
uint8_opts);
TensorWrapper input_tw, output_tw;
```

```
input_tw.set_rowwise_data(col_data.data_ptr, data_dtype,
{first_dim, last_dim});
output_tw.set_rowwise_data(data_transposed.data_ptr(), data_dtype,
{last_dim, first_dim});
nvte_transpose(input_tw.data(), output_tw.data(), stream);
```

Scale transposes (tiny tensors, ~0.3 MB each) were left using `.t().contiguous()` as their cost is negligible.

7.5 Result

Metric	Before	After	Float8Block
ms/step	159	**116**	113
MFU	42.7%	**58.6%**	60.8%
Speedup vs BF16	1.10x	**1.51x**	1.55x

The optimization eliminated 43 ms of overhead, bringing MXFP8 within 3 ms of Float8Block · within measurement noise. Correctness is unchanged (cosine 0.999611).

7.6 Remaining Minor Gap

The residual ~3 ms difference comes from the MXFP8 bidimensional quantization kernel being slightly more expensive than Float8Block's separate activation + quantize path. This is a TE CUDA kernel optimization opportunity, addressed in Section 8.

---

8. Further Optimization: Fused Quantize + Transpose Kernel

After the `nvte_transpose` optimization closed the major gap, the remaining ~3 ms overhead came from two sources: (1) `nvte_transpose` kernel launch overhead across 126 calls per step, and (2) redundant global memory round-trips (colwise data written by quantize kernel, then read and re-written by transpose kernel). This section describes fusing the transpose directly into the MXFP8 quantize kernel.

8.1 Approach: Transposed Colwise Output in Shared Memory

The existing bidimensional MXFP8 quantize kernel writes colwise FP8 data to shared memory in `[M, K]` layout (same as input), then TMA-stores to global memory. The key insight: by changing the shared memory write pattern, we can produce `[K, M]` layout directly · eliminating the need for any subsequent transpose.

Original colwise shmem write (per thread, handling one K-column):

```
shmem[buff*2048 + i*64 + tid]  // row i (M-dim), col tid (K-dim)
```

Transposed colwise shmem write:

```
shmem[buff*2048 + tid*32 + i]  // row tid (K-dim), col i (M-dim)
```

The transposed data is then TMA-stored to global memory with swapped coordinates, using a tensor map that describes the output as `[K, M]` (globalY=K, globalX=M, stride=M).

8.2 Implementation

Three files were modified:

1. `quantize_mxfp8.cuh` · **Kernel** + **Dispatch**

- Added `transpose_colwise` bool parameter to `quantize_mxfp8_kernel`
- Precomputed `colwise_out_base` and `colwise_out_stride` based on the flag (branch outside inner loop)
- Modified colwise scale indexing: `k * roundup(M/32, 4) + m_block` (transposed) vs

`m_block * stride + k` (original). The `roundup(M/32, 4)` is critical · using raw `M/32` caused a stride mismatch with the scale buffer allocation, producing NaN for tensors where `M/32` is not a multiple of 4
- Swapped TMA store coordinates: `(M_offset, K_offset)` instead of `(K_offset, M_offset)`
- Dispatch creates a transposed TMA tensor map with swapped global dimensions and tile sizes
- Only enabled for bidimensional scaling on SM 12.0 with non-square matrices (`rows != cols`)

2. `gemm.cpp` **· Detect Pre-Transposed Data**

- Added `already_transposed` check in `transpose_mxfp8_operand`: on SM 12.0, non-square MXFP8 colwise data is assumed pre-transposed by the quantize kernel
- When detected: swap shape dimensions `[M, K]` · `[K, M]` and relabel as rowwise (zero-cost, no data copy)
- Square matrices (`M == K`) fall back to explicit `nvte_transpose` because shape-based detection is ambiguous

3. Shape Metadata Strategy

The colwise data buffer's shape metadata remains `[M, K]` (unchanged from original allocation). Only the physical data layout changes to `[K, M]`. This avoids breaking `Tensor::shape()`, which returns colwise shape when rowwise data is absent · a common pattern in TE backward pass where tensors are split into colwise-only and rowwise-only for different GEMMs.

8.3 Technical Challenges

1. **CUDA kernel default parameters**: `__global__` function pointers lose default parameter values. All 3 kernel call sites (ROWWISE, COLWISE, BIDIMENSIONAL) must explicitly pass `false` for `transpose_colwise`
2. **Scale stride alignment**: MXFP8 scale buffers have inner dimensions rounded to multiples of 4 (`roundup(M/32, 4)`). The transposed scale write must use this rounded stride, not raw `M/32`. For `M=64`: `M/32=2` but buffer stride is `roundup(2, 4)=4`. Using stride 2 caused scale reads at wrong offsets · NaN gradients
3. **Shape vs layout disconnect**: Changing colwise tensor shapes (e.g., allocating as `[K, M]`) broke downstream code that uses `Tensor::shape()` to determine logical tensor dimensions. The solution: keep shape metadata unchanged, detect pre-transposition via architecture check in `gemm.cpp`

8.4 Results

Metric	Before (nvte_transpose)	After (fused)	Float8Block
ms/step (BS=4)	~116	~111-117	~113
MFU	~59%	~59-61%	~61%
Correctness (cosine)	0.999611	0.999611	0.999611

The fused transpose eliminates `nvte_transpose` kernel launches and temporary buffer allocations at GEMM time. The performance improvement is modest (~2-5 ms) because `nvte_transpose` was already fast after the previous optimization. The main benefit is reduced kernel launch overhead and memory allocation pressure, which may be more significant in larger models or multi-GPU settings.

---

9. Changes to train.py

The training script was modified from the original alint77/nanogpt-fp8 to support SM 12.0, flexible benchmarking, and the corrected optimizer.

8.1 Environment Variable Overrides

Three parameters are configurable via environment variables for benchmarking without editing code:

```
USE_FP8 = os.environ.get('USE_FP8', 'True').lower() != 'false'
batch_size = int(os.environ.get('BATCH_SIZE', '3'))
max_iters = int(os.environ.get('MAX_ITERS', '5000'))
```

8.2 Recipe Selection via Environment

The FP8 recipe is selected via the `RECIPE` environment variable (default: `MXFP8BlockScaling`):

```
RECIPE_NAME = os.environ.get('RECIPE', 'MXFP8BlockScaling')
if USE_NVFP4:
    recipe = NVFP4BlockScaling(...)
elif RECIPE_NAME == 'Float8BlockScaling':
    recipe = Float8BlockScaling()
elif RECIPE_NAME == 'DelayedScaling':
    recipe = DelayedScaling()
else:
    recipe = MXFP8BlockScaling()
```

Usage: `RECIPE=Float8BlockScaling BATCH_SIZE=4 MAX_ITERS=50 bash run.sh`

8.3 Optimizer: Muon to NorMuon

Switched from `Muon` to `NorMuon` (with `adjust_lr="rms_norm"`) to match the upstream repo:

```
optimizer_muon = NorMuon(param_groups,
    distributed_mesh=...,
    use_triton=True,
    weight_decay=0.01,
    cautious_wd=True,
)
```

The gains/biases/embeddings learning rate was updated from `lr=2e-3` to `lr=0.2` (NorMuon applies internal `rms_norm` scaling: `adjusted_lr = lr * 0.2 * sqrt(max(fan_out, fan_in))`).

Note: An older version of the `dion` library had a division-by-zero bug in `muon_update_post_orthogonalize` when `norm_U_new` was zero (from zero-initialized weights). This caused NaN in parameters after the first optimizer step. Fixed by updating dion to the latest commit (7452a58), which clamps `norm_U_new` to epsilon.

8.4 Other Changes from Original Repo (alint77/nanogpt-fp8)

Setting	Original `train.py`	This Fork
`n_layer`	12	20
`n_embd`	768	1280
`block_size`	1024	2048
`vocab_size`	50304	65536
`GPU_PEAK_TFLOPS`	2250 (B200)	404 (RTX 5090)
`USE_FSDP2`	True	False
`USE_DDP`	False	True
`USE_DOC_MASK`	True	(removed)
`USE_FAST_FSDP`	True	(removed)
Default recipe	DelayedScaling	MXFP8BlockScaling

---

10. Summary of All Changes

Transformer Engine (4 files)

| File | Change | Purpose | |---|---|---| | `pytorch/quantization.py` | Removed `if >= (12,0): return False` guard | Enable MXFP8 on SM 12.0 | | `pytorch/csrc/extensions/gemm.cpp` | Added `transpose_mxfp8_operand` lambda with fused-transpose detection; replaced `.t().contiguous()` with `nvte_transpose` (fallback) | Transpose MXFP8 columnwise data to TN layout; detect pre-transposed data from fused kernel | | `pytorch/csrc/quantizer.cpp` | `with_gemm_swizzled_scales &&= nvte_is_non_tn_fp8_gemm_supported()` | Disable pre-swizzled scales so gemm.cpp transpose gets natural-format scales | | `common/cast/mxfp8/quantize_mxfp8.cuh` | Added `transpose_colwise` parameter; transposed shmem write + TMA coordinates + scale indexing | Fuse colwise transpose into quantize kernel for SM 12.0 |

nanogpt-fp8 (train.py)

| Change | Detail | |---|---| | Env var: `USE_FP8` | Toggle FP8 on/off for BF16 baseline | | Env var: `BATCH_SIZE` | Override micro-batch size (default 3) | | Env var: `MAX_ITERS` | Override optimizer steps (default 5000) | | Env var: `RECIPE` | Select FP8 recipe (default MXFP8BlockScaling) | | Optimizer | Muon -> NorMuon with `weight_decay=0.01, cautious_wd=True` | | LR group 2 | `lr=2e-3` -> `lr=0.2` (NorMuon internally scales via rms_norm) | | Model config | 20 layers, 1280 hidden, 65536 vocab, 2048 seq_len | | `GPU_PEAK_TFLOPS` | 2250 -> 404 (RTX 5090) | | Distributed | FSDP2 -> DDP (single GPU) | | Removed features | `USE_DOC_MASK`, `USE_FAST_FSDP`, `USE_AC_BLOCKS`, `USE_AC_ATTENTION` |

Diagnostic Script (`diag_fwd_bwd.py`)

Standalone script that creates a single `te.Linear(1280, 1280)` layer, runs forward+backward in BF16, Float8BlockScaling, and MXFP8BlockScaling, and compares output/gradient cosine similarity. Used as regression test throughout development.

---

11. Conclusions

1. **MXFP8BlockScaling is fully functional on SM 12.0** after fixing the backward pass TN layout conversion. Weight gradient cosine similarity matches Float8BlockScaling at 0.999611.
2. **MXFP8 now matches Float8BlockScaling in throughput** after the `nvte_transpose` optimization and fused quantize+transpose kernel: ~111-117 ms/step (~59-61% MFU) vs ~113 ms/step (~61% MFU). Both achieve ~1.5x speedup over BF16. DelayedScaling achieves 1.41x.
3. **The original MXFP8 performance gap was caused by an inefficient transpose**, not the quantization kernel. PyTorch's generic `direct_copy_kernel_cuda` achieved only ~2% of peak memory bandwidth for the 280 GB/step of uint8 copies needed by backward GEMMs. Replacing it with TE's optimized `nvte_transpose` eliminated 43 ms of overhead. Fusing the transpose into the quantize kernel eliminated the remaining kernel launch overhead.
4. **All four recipes converge** on the 561M model. Float8Block and MXFP8 produce virtually identical loss curves (same GEMM kernels on SM 12.0). DelayedScaling converges slightly faster in early steps.
5. **Building from source was necessary** for MXFP8 support (the fix is not in any released TE version). The compilation process required working around cmake version issues, GCC 12 ICE bugs, CUDA architecture list quirks, and library conflicts.
6. **Both Float8BlockScaling and MXFP8BlockScaling are viable** on the RTX 5090, delivering identical numerical quality and near-identical throughput. The choice between them is now a matter of preference rather than performance.
7. **Fusing the transpose into the quantize kernel** is possible by modifying the shared memory write pattern and TMA store coordinates. The approach preserves shape metadata (avoiding downstream breakage) and uses architecture-based detection in gemm.cpp. The technique generalizes to any kernel that produces both rowwise and colwise FP8 output.

Appendix: Raw Benchmark Data (BS=4, 50 Steps)

A. Float8BlockScaling

Step	Loss	ms/step	MFU	Memory
2	10.072	112.0	60.95%	27.80 GB
5	8.583	111.9	61.01%	28.72 GB
10	9.870	118.8	57.45%	28.76 GB
15	6.937	112.5	60.68%	28.72 GB
20	7.172	114.1	59.82%	28.74 GB
25	6.592	113.9	59.92%	28.72 GB
30	6.423	113.6	60.11%	28.72 GB
35	6.316	118.4	57.66%	28.82 GB
40	6.271	115.8	58.94%	28.78 GB
45	6.254	111.0	61.52%	28.94 GB
49	6.054	114.0	59.88%	28.94 GB

B. MXFP8BlockScaling (after nvte_transpose optimization)

Step	Loss	ms/step	MFU	Memory
2	10.072	123.6	55.2%	28.07 GB
5	8.581	116.5	58.6%	28.07 GB
10	9.866	116.5	58.6%	28.07 GB
15	7.040	116.0	58.9%	28.07 GB
20	7.145	116.1	58.8%	28.07 GB
25	6.597	115.9	58.9%	28.07 GB
30	6.389	116.1	58.8%	28.07 GB
35	6.306	116.5	58.6%	28.24 GB
40	6.243	116.5	58.6%	28.24 GB
45	6.229	116.1	58.8%	31.25 GB
49	6.059	116.1	58.8%	31.25 GB

Before optimization: ~160 ms/step, 42.5% MFU

C. DelayedScaling

Step	Loss	ms/step	MFU	Memory
2	10.975	124.8	54.68%	26.86 GB
5	8.083	123.6	55.22%	27.41 GB
10	8.512	124.0	55.04%	27.41 GB
15	6.691	124.3	54.92%	27.41 GB
20	7.005	124.1	54.99%	27.41 GB
25	6.461	124.2	54.96%	27.41 GB
30	6.288	124.5	54.82%	27.41 GB
35	6.227	124.2	54.97%	27.41 GB
40	6.132	125.5	54.42%	27.41 GB
45	6.059	124.2	54.95%	27.89 GB
49	5.934	124.8	54.71%	27.89 GB

D. BF16 (no FP8)

Step	Loss	ms/step	MFU	Memory
2	10.060	174.4	39.14%	28.29 GB
5	8.574	174.2	39.18%	28.29 GB
10	9.859	174.6	39.09%	28.29 GB
15	7.049	174.9	39.03%	28.29 GB
20	7.060	175.1	38.97%	28.29 GB
25	6.558	175.3	38.94%	28.29 GB
30	6.341	175.3	38.93%	28.29 GB
35	6.258	175.3	38.94%	28.29 GB
40	6.213	175.6	38.88%	28.29 GB
45	6.138	175.3	38.93%	28.54 GB
49	5.998	175.5	38.90%	28.54 GB